CIS*3110 Operating Systems
Assignment #3
Due date: Mar 16, 2026 @ 11:59 PM EST

Instructor: Prof. Yulia Kotseruba

Winter 2026

This assignment is to be completed individually by each student. The assignment is worth 10% of the final grade.

Use of any AI tools is **not allowed** for any part of this assignment (including the README).

The assignment is due on **Monday Mar 16, 2026 @ 11:59 PM EST**. You may use grace days for this assignment. Late submissions, i.e. after the deadline + remaining grace days, will receive a grade of 0.

One of the conditions for passing the course is achieving over 50% average across 4 assignments. Please be mindful of this and prioritize work accordingly.

1 Assignment description

For this assignment, you will write a C program implementing paging with several cache replacement policies. Specifically, you will first implement a simple virtual-to-physical address translation with first-in-first-out (FIFO) page table replacement policy. You will then extend this program to include a TLB cache and two more replacement policies: optimal (MIN) and least-recently-used (LRU).

1.1 Basic paging

Basic paging was introduced in Lecture 8. We will use this as a warm-up exercise.

Assume that virtual memory (VM) size is fixed at 4KB and physical RAM is fixed at 1KB. The default page size is 128 bytes.

Your program will simulate MMU by translating virtual addresses to physical RAM addresses. A series of virtual addresses in hex format will be provided in a text file (one virtual address per line). Your program will read them one by one and output a corresponding physical address for the given sizes of the VM, RAM, and page.

As discussed in class, virtual address consists of two components: virtual page number (VPN) and offset.

To perform address translation, your program will need the following:

- A number of virtual pages and physical frames available in the system;
- A page table: indexed by the virtual pages in the system and containing the corresponding physical pages;
- Two masks: one mask to extract the VPN and another to extract an offset into the page. For this, you need to calculate how many bits are needed for VPN and for the offset.
- Page replacement policy: since physical RAM is smaller than virtual memory, all virtual pages cannot fit in RAM. Once RAM fills up, some frames will need to be evicted to make room for new pages. You will implement the first-in-first-out (FIFO) policy for page replacement.

Pseudocode:

```
for each VADDR

  compute VPN and OFFSET from VADDR using bitwise operations

  if VPN index >= number of virtual pages
    VADDR is invalid
    return NULL

  if FRAME for VPN in PAGE_TABLE
    return FRAME

  else // page fault

    if RAM is full
      evict FRAME following the FIFO policy

    load the PAGE into free FRAME
    update the PAGE_TABLE

    combine FRAME with OFFSET to get PADDR

  return PADDR
```

In addition to the address, your program should compute and output the following statistics:

- Total accesses: number of vaddresses accessed (not counting the invalid vaddresses);
- Page faults: number of page faults;
- Fault rate: page faults divided by the total accesses.

1.2 Paging with TLB

For this part of the assignment, you will extend the implementation of paging with Translation Lookaside Buffer (TLB) which will cache previous valid address translations.

Memory parameters will remain the same. Assume that virtual memory (VM) size is fixed at 4KB and physical RAM is fixed at 1KB. The default page size is 128 bytes. The default TLB size is 4 entries.

The algorithm will remain mostly the same as well. The only change is that before checking the page table, your program should first check whether the VPN is in TLB. If the VPN is there (TLB hit), then all it needs to do is grab the cached frame and combine it with the vaddr offset. Otherwise, a TLB miss occurs and the program will check the page table (and deal with page fault if needed). It will then need to update the TLB cache. If there is no free entry in the TLB cache, one entry should be removed following the FIFO policy.
Here is the modified pseudocode:

```
for each VADDR
  compute VPN and OFFSET from VADDR using bitwise operations

  if VPN index >= number of virtual pages
    VADDR is invalid
    return NULL

  if VPN index in TLB  // TLB hit
    return FRAME

  else //TLB miss

    if FRAME for VPN in PAGE_TABLE
     return FRAME

    if FRAME for VPN not in PAGE_TABLE  // page fault

    if RAM is full
      evict FRAME following FIFO policy

    load PAGE into free RAM FRAME
    update PAGE_TABLE

    update TLB
      evict TLB entry following FIFO policy if needed

    combine FRAME with OFFSET to get PADDR

  return PADDR
```

In addition to the address, your program should compute and output the following statistics:

- Total accesses: number of vaddresses accessed (not counting the invalid accesses);
- Page faults: number of page faults;
- Fault rate: page faults divided by the total accesses.
- TLB hits: number of TLB hits
- TLB misses: number of TLB misses
- TLB hit rate: TLB hits divided by total accesses.

1.3 MIN and LRU replacement strategies

Add two more page replacement strategies to your program:

- Optimal strategy (MIN): remove an item which will be used farthest into the future. In case of tie, remove the oldest item;
- Least recently used (LRU): remove the least recently used item, i.e. an item that has not been used the longest.

You are free to use any data structures for implementing these additional strategies, however, no external libraries are allowed.

Your program should be able to use MIN, LRU, or FIFO in case either a TLB miss or page fault occurs. In other words, you will not be required to apply MIN for TLB miss and LRU for page fault, the replacement algorithm will be the same for both.

The addressing approach and output statistics will remain the same for this part of the assignment.

2 Deliverables

There is a minimal starter code for this assignment to streamline automated tests and to help you structure your code:

- `paging.c`: contains a stab for implementing the basic paging algorithm with some constants and argument parsing;
- `paging_tlb_fifo.c`: once you have completed the `paging.c`, copy the code into `paging_tlb_fifo.c` and extend it with additional requirements;
- `paging_tlb.c`: once you have completed the `paging_fifo.c`, copy the code into `paging_tlb.c` and extend it with additional requirements;

  Note: Initially all source files contain the same code to pass the build stage and generate the report.
- `Makefile`: this makefile is to be used to compile your code. It expects the source code and headers to be in the `src` directory.

  The provided `Makefile` will compile your code into three executables `paging`, `paging_tlb_fifo`, and `paging_tlb`. It also has a debug target if you need to debug your code.

  You may change the Makefile if you are not developing on Linux/Mac, however, the autograder will use this Makefile to build your code. Therefore, you should not link any libraries or add more source files.

Similar to A1 and A2, you will use the provided GitLab repository for development. Code submitted through other means, e.g. via email, on CourseLink, via a link to external repository, will not be graded.

3 Setting up

The setup for A3 is the same as for A1 and A2. The only change is that now in the `.gitlab-ci.yml` file you need to provide the address of the A3 repo. The rest of the steps are the same and are repeated below for convenience.

1. Clone the A3 repo from GitLab CIS3110S01W26.

   In your terminal, go to the directory where you plan on developing the assignment using `cd` command.

   Then clone the repository:

   `git clone https://gitlab.socs.uoguelph.ca/cis3110s01w26/YOUR_USERNAME/A3.git`

   Replace `YOUR_USERNAME` with your username. It should be the same as your official uoguelph.ca email.

   After this step, you will have A3 folder on your local computer where you can start developing your code.

2. Download `A3_starter.zip` from CourseLink, unzip it, and copy the files into your A3 directory. **Do not copy the A3_starter directory itself into A3, just copy all files and folders, including the hidden yml file.**

3. Open `.gitlab-ci.yml` and locate lines 11 and 12 with TODO comments;

4. Change line 10 to your username (which is the same as your uoguelph.ca email), e.g. `STUDENT_ID: "marissa"`

5. Change line 11 to your repo for this assignment, e.g. `STUDENT_REPO: "marissa/A3"`.

6. Do not add anything else, e.g. `A3.git`, `assignment/A3/`, `cis3110/A3`, etc. Just your username and A3 separated by the forward slash

7. Make sure that `.gitlab-ci.yml` is a hidden file (its name should start with the dot). If it is not, rename if using `mv gitlab-ci.yml .gitlab-ci.yml`.

8. Add all files to git:

```
git add .gitlab-ci.yml
git add src
git add Makefile
git add README.md
git commit -am "inital commit"
git push
```

Every time you push your code into the repo, it will trigger the autograder which will test your code and output a feedback report. If it does not, consult the Autograder FAQ first (Section 4.4). If the issue persists, reach out to course staff.

To see the grading report, go to your A3 repo in gitlab.socs.uoguelph.ca/cis3110s01w26/ and select a branch called A3_feedback. There, you will find timestamped feedback files, which contain a list of all tests that your program was tested on, points received for each test, and errors your program produced that would be useful for debugging.

IMPORTANT: **Make sure that your best submission is the latest pushed to the repo.** The new autograder will also include remaining grace days in your report and will not accept submissions after the deadline + remaining grace days.

4 Grading

4.1 Academic integrity

As a reminder, you are expected to submit your own work. After the submission deadline, all repositories will be examined. Similar submissions or those with suspected use of AI tools will be given a grade of 0 and formally investigated for academic misconduct. The grade will be adjusted only if the student successfully proves that it is their work.

4.2 Grade distribution

The assignment will be graded out of 100 points and is worth 10% of the course final grade.

- **[90 points]** – passing all automated tests;
- **[10 points]** – README.md.

Note: achieving 50% or more on the assignments is a requirement for passing the course. Start working on the assignment early and be strategic to maximize the grade.

Here are several tips that will help you achieve this goal:

- Start working on the assignment ASAP to get familiar with the problem and get autograder feedback. This assignment is not difficult, but it has more moving parts than A1 and A2;
- Design your code so that it is easier to extend it with additional replacement strategies. The best approach is to have separate functions to manage RAM and TLB replacement;
- Commit often to catch bugs early through autograder feedback.
- Write your README as you progress through the assignment rather than leaving it at the very end.

4.3 Automated tests

You do not need to develop this code remotely on the SoCS Linux servers or using a docker. All code for the assignment can be written and tested on your local machine.

Once you push changes to the provided GitLab repository, provided that it is set up correctly, a number of tests will be automatically run on your code and a report will be produced. This report will appear in the A3_feedback branch in your repo shortly after the push. Each submission will be run through the same tests using the same docker environment to ensure consistency.

4.3.1 Build stage

The autograder will check the integrity of the Makefile first. If the file has not been modified, `make clean & make` commands will be run to rebuild the project and produce three executables: `paging`, `paging_tlb_fifo`, and `paging_tlb`. If the executables fail to build at this stage, the submission receives a grade of 0.

4.3.2 Test stage

If the executables build successfully, they will be tested on X different inputs.

- **Command format.** The executables will be run using the following commands:

  `./paging [-p page_size] <input.txt>`

  `./paging_tlb_fifo [-p page_size] [-t tlb_size] <input.txt>`

  `./paging_tlb [-p page_size] [-t tlb_size] [-s strategy] <input.txt>`

  The following flags are expected:

  `-p` — page size in bytes, default=128;

  `-t` — tlb size, default=4 entries;

  `-s` — cache replacement strategy (FIFO, MIN, LRU) for **both** page faults and TLB misses, default=FIFO.

  Note: all flags are optional, if they are not provided, defaults should be used. If any additional or invalid parameters are provided, the program should exit with failure and print an appropriate message to `stdout`.

- **Input format.** Input is a text file, with each line representing a virtual address in hex format. *Note: all inputs will be valid, there will be no empty or malformed lines, so your code does not need to check for input integrity.*

```
0x0000
0x0000
0x0200
0x0200
0x0400
0x0400
0x0600
0x0600
0x0800
0x0800
0x0a00
0x0a00
0x0c00
0x0c00
0x0e00
0x0e00
```

- **Expected output.** The autograder will expect the trace of execution and statistics as shown in the output samples. You may add other print statements for debugging purposes, however, they should be removed prior to testing.

  For example `paging` should produce the following output for the input above:

```
VM_SIZE 4096B
RAM_SIZE  1024B
PAGE_SIZE 128B
NUM_VPAGES  32
NUM_FRAMES  8
PAGE_POLICY FIFO

   0 [vaddr] 0x0000 -x->  [paddr] 0x0000
   1 [vaddr] 0x0000 --->  [paddr] 0x0000
   2 [vaddr] 0x0200 -x->  [paddr] 0x0080
   3 [vaddr] 0x0200 --->  [paddr] 0x0080
   4 [vaddr] 0x0400 -x->  [paddr] 0x0100
   5 [vaddr] 0x0400 --->  [paddr] 0x0100
   6 [vaddr] 0x0600 -x->  [paddr] 0x0180
   7 [vaddr] 0x0600 --->  [paddr] 0x0180
   8 [vaddr] 0x0800 -x->  [paddr] 0x0200
   9 [vaddr] 0x0800 --->  [paddr] 0x0200
  10 [vaddr] 0x0a00 -x->  [paddr] 0x0280
  11 [vaddr] 0x0a00 --->  [paddr] 0x0280
  12 [vaddr] 0x0c00 -x->  [paddr] 0x0300
  13 [vaddr] 0x0c00 --->  [paddr] 0x0300
  14 [vaddr] 0x0e00 -x->  [paddr] 0x0380
  15 [vaddr] 0x0e00 --->  [paddr] 0x0380

Total accesses : 16
Page faults  : 8
Fault rate   : 50.00%
```

  Output formatting. Output always starts with the header. What is printed will change as you progress through the assignment:

```
VM_SIZE 4096B
RAM_SIZE  1024B
PAGE_SIZE 128B
NUM_VPAGES  32
NUM_FRAMES  8
PAGE_POLICY FIFO // paging_tlb also has LRU and MIN options
TLB_SIZE  64 // only for paging_tlb_fifo and paging_tlb
TLB_POLICY  FIFO // only for paging_tlb_fifo and paging_tlb
                 // paging_tlb also has LRU and MIN options
```

Your programs will translate virtual addresses to physical addresses based on the given virtual and physical memory parameters. For each line of input, your program will output line number (padded with 6 spaces), virtual address (padded with 4 zeros in hex) and its physical translation (padded with 4 zeros in hex). Different types of arrows between them indicate whether the translation incurred a page fault or a TLB hit (if applicable). If the address is invalid, four `X`s should be shown instead.

```
   0 [vaddr] 0x0000 --->  [paddr] 0x0000 // no page faults
   1 [vaddr] 0x0000 -x->  [paddr] 0x0000 // page fault
   2 [vaddr] 0x0000 ===>  [paddr] 0x0000 // TLB hit
   3 [vaddr] 0xFFFF XXXX  Invalid address // invalid address
```

Basic stats are printed after all translations are done. TLB hits/misses will only be printed for `paging_tlb_fifo` and `paging_tlb`.

```
Total accesses : 64
Page faults  : 32
Fault rate   : 50.00%
TLB hits    : 32 // only for paging_tlb_fifo and paging_tlb
TLB misses   : 32 // only for paging_tlb_fifo and paging_tlb
TLB hit rate  : 50.00% // only for paging_tlb_fifo and paging_tlb
```

4.4 Autograder FAQ

- *The autograder feedback report is not generated after pushing the code.*

  Check the following:

  - Check that the .gitlab-ci.yml file is committed to the repo;
  - Make sure the .gitlab-ci.yml is a hidden file (filename begins with the .), rename if needed;
  - Make sure the username is filled in correctly and matches your `uoguelph.ca` email;
  - Make sure you are using .gitlab-ci.yml for A3, not for one of the previous assignments.
    You can find the correct yml file in the `A3_starter.zip`. It is a hidden file, so it may not be visible in the file managers, but you should be able to locate it via command line;
  - Make sure you did not commit the `A3_starter` directory to the repo, only the files in it.

  If you are still not getting feedback, please reach out via course email and we will look at the issue.

- *One of the tests is timing out.*

  The most likely reason for hitting a timeout is infinite loop and/or leaking memory. It is very unlikely due to the inefficient code since the timeout value is about 100x time needed to process any of the inputs.

  If this is happening, comment out the code until you are able to get feedback and then gradually add code back to find the area which is causing trouble.

- *One of the tests is not passing.*

  Autograder is designed to cover most edge cases, however, it may miss some bugs. Typically they manifest themselves in stress tests.

If you believe there is a bug in the autograder, let the course staff know. Please describe the issue in detail and how to reproduce it. E.g. which test case is failing, the output your program produces, and what did you expect to see instead.

If we can confirm it is on our side, we will push an update.

- *Number of grace days remaining.*

  The autograder for A3 will show remaining grace days in the report after the grace period for A2 finishes on Feb 27th.

  As a reminder, there are 4 grace days per term. If you used any for A1 and A2, there will be fewer left for later assignments. The autograder will not run on submissions after deadline + remaining grace days.

4.5 README

Assume that you are writing a README for a project hosted on a public repository that anyone can access. You may assume prior knowledge of programming and git (i.e., you do not need to explain how C, Makefiles, or git work). Anyone reading your repo should be able to understand the purpose of the project, build, and run your code by following the provided instructions.

You may refer to A1 and A2 instructions for more information on READMEs.

Table 1: A rubric for evaluating README.md files. Note that this time the rubric includes formatting in addition to contents.

Grade & Justification	
Excellent 10	All required and optional sections are complete and correct. Instructions are clear, detailed, and sufficient to build and run the code on Linux and other major operating systems. Example inputs/outputs and demo script are provided. Formatting: neat and clear formatting throughout. Commands are shown with backticks, code is quoted correctly. Lists (if any) are rendered correctly.
Good 8	All required sections are provided and are sufficiently detailed. It is possible to use the code with minor tweaks to the instructions. Example inputs are provided but are not complete/detailed. Formatting: minor issues, e.g. missing backticks for commands or code not in code blocks.
Marginal 6	Incomplete and/or poorly organized README. Assignment instructions are copied verbatim or with very minimal changes. The instructions are incomplete and/or require major tweaks to work. Formatting is messy, the document is difficult to read.
Unsatisfactory 4	Too brief, vague, and/or unclear README. Instructions are not provided or are incorrect.No input/output examples. Formatting is messy, the document is difficult to read.
No grade 0	No submission or unmodified README

ADVANCED: Docker setup

This is **highly optional** and may take some time to set up the first time. **Do not attempt this** if you are short on time and return to this later if you are curious.

If you are still reading, instructions below outline how to test your code locally using a docker container that we use to run your code on the Linux server.

This is not a part of the class material, therefore technical support for docker issues will not be prioritized. However, if you spend some time learning about dockers, you may find them quite useful in the future.

You will first need to install Docker following instructions here `https://docs.docker.com/desktop/`. The setup is different depending on the OS you are using, there are instructions for most common OSs on the webpage. Note that you might need to set up additional permissions and environment variables to use docker.

On Linux, it is recommended to add yourself to a `docker` group to avoid using `sudo` to run dockers, as explained in https://docs.docker.com/engine/install/linux-postinstall/.

On Mac OS, you will need to add a path to Docker to your environment variable in order to be able to use it. I just add a line `export PATH="/Applications/Docker.app/Contents/Resources/bin:$PATH"` to `.zshrc` or `.bashrc` file in my home directory.

Once you are able to call `docker` from your command line, you will need to download the Docker image from DockerHub[1] using this command in your terminal:

```
docker pull socsguelph/cis3110:w26
```

Then `cd` to your A3 directory and run this command:

```
docker run --rm -it \
 --name cis3110 \
 --mount type=bind,source=`echo $pwd`,target=/CIS3110/A3 \
 socsguelph/cis3110:w26
```

If your docker is installed correctly, your terminal prompt will change to something like `socs@36058c626e8`. This means you are now inside the Docker container running Linux (run `uname -r` to check).

Your A3 directory will be mounted in `/CIS3110/A3` (note the forward slash). If you go to that directory (`cd /CIS3110/A3`), you can run code in exactly the same environment as our Linux servers.

Note that because your A3 is mounted inside the docker container, any changes you make will be reflected in your local file system and vice versa.

If you have any suggestions on how to improve these instructions, please let me know.

[1] There are a bunch of other images available on `https://hub.docker.com/u/socsguelph`.